

Integrating ESP32 with AWS IoT Core

Sensor readings in the cloud and over-the-air (OTA) firmware updates · ESP32-S3 + ESP-IDF 5.5 · region eu-central-1

This note covers two things: **(A)** how to set up AWS so an ESP32 can send sensor readings to the cloud, and **(B)** how to add over-the-air firmware updates on top of that. The AWS steps are described the way they look in the web console. The code snippets only show the places that talk to the cloud.

The examples use these names: thing `esp32s3-htu21d`, bucket `htu21d-ota-bucket`, region `eu-central-1` (Frankfurt). `<ACCOUNT_ID>` is the 12-digit AWS account number, shown when you click the account name in the top right corner of the console.

Glossary

Term	What it is
AWS IoT Core	Amazon's service that acts as an MQTT broker. The device connects to it, and all data passes through it.
Thing	An entry in AWS that represents one device. It has a name AWS uses to recognise the device.
Device certificate	A pair of files (certificate + private key) the device uses to prove who it is. The equivalent of a username and password.
Policy	The list of things a device is allowed to do: what it may connect to, which topics it may publish to and which it may listen on.
Topic	The address of an MQTT message, e.g. <code>sensors/esp32s3-htu21d/data</code> .
S3 / bucket	File storage in the cloud. A bucket is a single container in that storage.
IAM role	A set of permissions one AWS service can temporarily take on in order to do something on your behalf.
Job	An instruction sent to a device, e.g. "install version 1.0.1". The device receives it over MQTT and reports back a status.
Presigned URL	A signed link to a private file in S3, valid for a limited time. Anyone holding it can download that one file.

Part A · Sending sensor data from the ESP32

End result: the ESP32 wakes up, connects to WiFi, reads the sensor and publishes the result to AWS as an MQTT message, e.g. `{"t":23.45,"h":58.20}`.

A1. Choosing a region

Everything has to be done in one region. The region is picked from the list in the top right corner of the console, next to the account name. This note uses Europe (Frankfurt) `eu-central-1`.

The region cannot be changed later without recreating everything, because the thing, the certificates and the broker address all belong to a region.

A2. Creating the device (thing)

1. Type **IoT Core** into the console search bar and open the service.
2. In the left menu: **All devices** → **Things** → **Create things**.
3. Choose **Create single thing**.
4. In **Thing name** enter the device name, e.g. `esp32s3-htu21d`. The remaining fields can be left alone (leave Device Shadow on *No shadow*).

Important: remember this name. It must be identical to the client ID in the firmware, and for OTA updates it also becomes part of the topic addresses. A mismatch is the most common cause of dropped connections.

A3. Device certificate

On the next screen of the wizard choose **Auto-generate a new certificate**. Once created, AWS offers these files for download:

File	What it is for
Device certificate (<code>.pem.crt</code>)	Identifies the device to AWS.
Private key (<code>.pem.key</code>)	Proves the device owns that certificate.
Amazon Root CA 1	Lets the device check that it is talking to the real AWS.

Note: the private key cannot be downloaded a second time. If it is lost, a new certificate has to be generated. Keys must never go into a repository — add the folder holding them to `.gitignore`.

At the end of the wizard the certificate must be **activated** and attached to the thing. You can check this under **Security** → **Certificates** — the status should be *Active*.

A4. Policy

Without a policy AWS drops the connection right after login. Go to **Security** → **Policies** → **Create policy**, give it a name and switch the editor to **JSON**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:client/esp32s3-htu21d"
    },
    {
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topic/sensors/esp32s3-htu21d/data"
    }
  ]
}
```

```
]
}
```

What this means:

- `iot:Connect` — permission to log in to the broker under this client ID.
- `iot:Publish` — permission to send messages to this one topic.
- If the device also needs to receive messages, add `iot:Subscribe` and `iot:Receive`. For `Subscribe` the resource uses `topicfilter/` instead of `topic/` — that is how AWS tells the two apart.

Once saved, the policy has to be attached to the certificate: **Security** → **Certificates** → (pick the certificate) → **Attach policy**.

A5. Broker address (endpoint)

In IoT Core open **Settings** and copy the **Device data endpoint**. It looks like this:

```
xxxxxxxxxxxxx-ats.iot.eu-central-1.amazonaws.com
```

This is the address the device connects to. It is different for every account.

A6. Settings on the ESP32 side

The three downloaded files go into the project (e.g. `main/Certs/`) and are compiled into the firmware. In `main/CMakeLists.txt`:

```
idf_component_register(SRCS "main.c" "WiFi/wifi.c" "Sensor/sensor.c" "MQTT/mqtt.c"
                        INCLUDE_DIRS "." "Sensor"
                        EMBED_TXTFILES "Certs/AmazonRootCA1.pem"
                                      "Certs/device.pem.crt"
                                      "Certs/private.pem.key")
```

Credentials live in a separate file (`secrets.h`) excluded from the repository:

```
#define WIFI_SSID      "..."  
#define WIFI_PASS      "..."  
#define AWS_IOT_ENDPOINT "xxxxxxxxxxxxx-ats.iot.eu-central-1.amazonaws.com"  
#define AWS_IOT_CLIENT_ID "esp32s3-htu21d" // must equal the thing name
```

MQTT client configuration:

```
const esp_mqtt_client_config_t cfg = {  
    .broker.address.uri = "mqtt:// " AWS_IOT_ENDPOINT ":8883",  
    .broker.verificatio...certificate = aws_root_ca_pem,           // verifies AWS  
    .credentials.client_id = AWS_IOT_CLIENT_ID,                   // must match the policy  
    .credentials.authentication.certificate = device_cert_pem,  
    .credentials.authentication.key = private_key_pem,  
};
```

`mqtt://` and port 8883 mean an encrypted connection in which both sides check each other: AWS verifies the device certificate, and the device verifies AWS using the Amazon Root CA.

A7. Checking that the data arrives

1. In IoT Core open **Test → MQTT test client**.
2. Under **Subscribe to a topic** enter the topic, e.g. `sensors/esp32s3-htu21d/data` (or `#` to see everything).
3. Power up the device. Messages should start appearing in the list.

Common problems

Symptom	Cause
Connection drops immediately, with no error message	The policy. When permissions are missing, AWS simply closes the connection. Check that the client ID and the topic match the policy character for character.
TLS / handshake error	Wrong certificate, an inactive certificate, or the Amazon Root CA missing on the device.
No connection, timeout	Wrong endpoint, or an endpoint from a different region.

A8. What happens to the data next (optional)

MQTT messages are not stored anywhere by themselves. To do something with them you create a rule: **Message routing → Rules → Create rule**. A rule selects messages with an SQL query, e.g.:

```
SELECT t AS temperature, h AS humidity FROM 'sensors/esp32s3-htu21d/data'
```

and forwards them — for example to CloudWatch (charts on a dashboard), to a database, or to a Lambda function.

Part B • OTA updates

How it works: the new firmware sits in a private S3 bucket. A job in AWS IoT tells the device which version to install and where to get it. The device downloads the file over HTTPS, installs it into the second partition and reboots. If the new version does not work, it falls back to the previous one.

B0. What the device needs

These are prerequisites on the ESP32 side — without them the AWS configuration is useless.

Item	Setting
Flash size	The value in <code>menuconfig</code> must match reality. Read it from the chip with <code>esptool.py -p COM7 flash_id</code> (the <i>Detected flash size</i> line).
Partition table	Two application partitions plus <code>otadata</code> . A custom <code>partitions.csv</code> in the project root, selected in <code>menuconfig</code> as <i>Custom partition table CSV</i> .
Rollback	<code>CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE</code> turned on. Without it the rollback functions do nothing.
Version number	<code>set(PROJECT_VER "1.0.0")</code> in the top-level <code>CMakeLists.txt</code> , before <code>include(...)</code> . The device uses this number to tell whether a job refers to a newer version.
Main task stack	About 8192 bytes. An HTTPS download does not fit in the default 3584 and ends in a crash.

An example `partitions.csv` (16 MB flash):

```
# Name,   Type, SubType, Offset, Size
nvs,      data, nvs,      ,      0x6000
otadata,  data, ota,      ,      0x2000
phy_init, data, phy,      ,      0x1000
ota_0,    app,  ota_0,    ,      2M
ota_1,    app,  ota_1,    ,      2M
```

A new image always goes into the free partition, so the old version stays untouched. `otadata` records which partition to boot from. `nvs` is used, among other things, to remember a version that turned out to be faulty.

In the firmware the download is handled by `esp_https_ota`. Two things differ from a plain local server setup:

```
esp_http_client_config_t http_config = {
    .url = url,                                     // the link comes from the job
    .cert_bundle_attach = esp_cert_bundle_attach, // built-in certificate bundle (for S3)
    .buffer_size_tx = 4096,                         // the link is 1-2 kB, the default 512 is too
    small
};
```

`esp_cert_bundle_attach` hooks up a bundle of a few hundred certificate authorities compiled into the firmware (option `CONFIG_MBEDTLS_CERTIFICATE_BUNDLE`). That lets the device verify S3 without adding any certificate files of its own.

B1. Extending the policy with job topics

The device talks about jobs on the reserved topics `$aws/things/<name>/jobs/...`. Without permission AWS drops the connection. Under **Security** → **Policies** open the policy, click **Edit active version**, switch to JSON and add two statements:

```
{
  "Effect": "Allow",
  "Action": ["iot:Publish", "iot:Receive"],
  "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topic/$aws/things/esp32s3-htu21d/jobs/*"
},
{
  "Effect": "Allow",
  "Action": "iot:Subscribe",
```

```
"Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topicfilter/$aws/things/esp32s3-htu21d/jobs/*"
}
```

When saving, tick **Set the edited version as the active version**.

B2. S3 bucket for the firmware

1. In the console open **S3 → Create bucket**.
2. **Region**: the same one as IoT Core.
3. **Bucket name**: lowercase letters, digits and hyphens only, e.g. `htu21d-ota-bucket`. The name must be globally unique, because it is part of the address.
4. **Block all public access**: leave it ticked. The firmware should not be publicly available — the device will be given a signed link instead.
5. **Default encryption**: leave *SSE-S3*. With KMS encryption the role from step B3 would need extra permissions for the keys.

B3. IAM role for signing links

The bucket is private, and the device has no permissions in S3 at all — it only holds a certificate for IoT Core. So the link to the file is prepared by AWS IoT, using a role that is allowed to read from the bucket.

Part 1: creating the role

1. **IAM → Roles → Create role**.
2. **Trusted entity type**: *Custom trust policy*. Paste this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "iot.amazonaws.com" },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": { "aws:SourceAccount": "<ACCOUNT_ID>" },
        "ArnLike": { "aws:SourceArn": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:*" }
      }
    }
  ]
}
```

This document says who may use the role: the AWS IoT service, but only from your account and from this region. The conditions matter, because every AWS customer uses the same IoT service — without them someone else could point at your role from their own job.

On the permissions screen select nothing, give the role a name (e.g. `iot-ota-s3-presign-role`) and create it.

Part 2: permission to read the bucket

Open the new role, **Add permissions → Create inline policy**, JSON mode:

```
{
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::htu21d-ota-bucket/*"
  }
]
}

```

So the role can only download files, and only from this one bucket.

How the pieces connect: the role is not attached to the bucket — the bucket knows nothing about it. The role has the bucket written into its own permissions, and you select the role when creating a job. The same role serves every future update, as long as the files stay in the same bucket.

B4. Release files

One release consists of two files. The firmware:

1. Change the version number in `CMakeLists.txt`, e.g. `set(PROJECT_VER "1.0.1")`.
2. Build it: `idf.py build`.
3. Copy `build/<project_name>.bin` under a name carrying the version, e.g. `firmware-1.0.1.bin`.

And the job document, e.g. `job-1.0.1.json`:

```

{
  "version": "1.0.1",
  "url": "${aws:iot:s3-presigned-url:https://s3.eu-central-1.amazonaws.com/htu21d-ota-bucket/firmware-1.0.1.bin}"
}

```

This is a plain text file written by hand. Its contents are up to you — the device just has to know how to read them. The `${aws:iot:s3-presigned-url:...}` placeholder is where AWS inserts a fresh signed link at the moment the device asks for the job.

Upload both files to the bucket: **S3** → (bucket) → **Upload** → **Add files**.

B5. Creating the job

1. **IoT Core** → **Remote actions** → **Jobs** → **Create job**.
2. Choose **Create a custom job**.
3. **Name:** e.g. `ota-1-0-1`. Dots are not allowed, hence the hyphens.
4. **Job targets** → **Things**: select your device.
5. **Job document:** *From file*, then point at `job-1.0.1.json` in the bucket.
6. Tick **Pre-sign resource URLs**. AWS should detect the firmware file address and show it underneath.
7. **Pre-signing role:** pick the role from step B3.
8. **URL expiration:** the default 60 minutes is plenty — downloading 1 MB takes a few seconds.
9. **Job configuration:** *Snapshot job*, meaning a one-off job for the selected devices.
10. **Submit**. From this moment the job is live and waits for the device to ask for it.

Note: the job document is copied from the JSON file at creation time. Changing that file in the bucket afterwards has no effect on an existing job.

B6. How an update runs

1. The device wakes up, connects to WiFi and AWS, and publishes its sensor reading.
2. It asks for a job by publishing to `$aws/things/<name>/jobs/start-next`.
3. AWS prepares a signed link (using the role) and sends the job document back on `.../start-next/accepted`: job ID, version number and link.
4. The device compares the version with its own. If it differs, it downloads the image from the link into the free partition and reboots.
5. After the reboot the new version starts and tries to connect to AWS. A successful connection is the proof that the version works — only then is it confirmed permanently.
6. The next time it asks for a job, the device sees it is already running the job's version and reports `SUCCEEDED` on `.../jobs/<jobId>/update`. The job is finished.

Progress is visible in the console: **Jobs** → (job) → **Job executions**; the status moves from *Queued* through *In progress* to *Succeeded*. The page does not refresh by itself.

When something goes wrong

Situation	What the device does
The new version cannot reach AWS	The bootloader returns to the previous version, still intact in the other partition. The faulty version is blacklisted in NVS, and the job is reported as <code>FAILED</code> .
Download interrupted (network, expired link)	The job stays open (<code>IN_PROGRESS</code>). On the next wake-up the device tries again with a fresh link.
The firmware file version does not match the job version	The device reports <code>FAILED</code> — a file with a different version number than the job states was uploaded to the bucket.

Important: a version that was once rejected is remembered and will not be installed again. A fix has to be released under a new number, e.g. 1.0.2 rather than a corrected 1.0.1.

B7. Releasing the next version

A job cannot be reused: once it reports `SUCCEEDED` it is closed and the device will not receive it again. The contents of an existing job cannot be swapped either. So every release means a new job:

1. New version number in `CMakeLists.txt` and `idf.py build`.
2. A copy of the JSON file with the version bumped in both places (the `version` field and the firmware file name).
3. Upload both files to the bucket.
4. A new job in the console, e.g. `ota-1-0-2`.

Old jobs do not need deleting — they stay as a history of updates. The clicking can be shortened: the **Save as a job template** button on the job page stores the settings (role, job type), and for full automation a single AWS CLI command, `aws iot create-job`, does the same in a script together with the upload to S3.

B8. Quick checklist

Symptom	Where to look
The job stays in <i>Queued</i>	The device is not asking for jobs at all: it is off, has no WiFi, or the policy does not cover the <code>jobs/*</code> topics.
The connection drops after adding job support	The policy: missing <code>topicfilter/</code> for <code>Subscribe</code> , or a thing name that does not match the client ID.
Download fails with HTTP 403	The role lacks <code>s3:GetObject</code> for that bucket, or <i>Pre-sign resource URLs</i> was not ticked on the job.
Error right at the start of the download	The buffer for the link is too small (<code>buffer_size_tx</code>), or the certificate bundle is missing.
The device reboots in a loop	The main task stack is too small, or the new version cannot reach AWS (in which case rollback handles it by itself).

Appendix: MQTT topics used for updates

Topic	Purpose
<code>\$aws/things/<name>/jobs/start-next</code>	The device asks for the next job.
<code>\$aws/things/<name>/jobs/start-next/accepted</code>	The AWS reply with the job document and the link.
<code>\$aws/things/<name>/jobs/start-next/rejected</code>	A refusal (e.g. a malformed job document).
<code>\$aws/things/<name>/jobs/<jobId>/update</code>	The device reports its status: <code>SUCCEEDED</code> or <code>FAILED</code> .

Sources: AWS IoT documentation (Managing jobs, Jobs device MQTT API, Create and manage jobs by using the console), Amazon S3 (Creating a general purpose bucket, bucket naming rules), ESP-IDF 5.5 (Partition Tables, OTA / App rollback, ESP x509 Certificate Bundle, ESP HTTP Client, ESP-MQTT).